



Guide technique

OPESc+

Le code, la maintenance et le déploiement

Version 2.1

Division des Analyses et des Politiques Économiques

MINEPAT

Septembre 2026

Table des matières

1	Vue d'ensemble	4
1.1	Ce que fait le programme	4
1.2	Les quatre étages	4
1.3	Le catalogue, pivot du programme	4
1.4	Pourquoi R et Shiny	4
2	Comment fonctionne Shiny	4
2.1	Deux fonctions, une conversation	4
2.2	La réactivité	5
2.3	Les sorties	5
2.4	Les modules	5
2.5	Le cycle d'une session	6
2.6	Les erreurs qui reviennent	6
2.7	Ce que Shiny ne fait pas	6
3	La structure golem	6
3.1	Un paquet plutôt qu'un script	6
3.2	Les fichiers d'ossature	7
4	Les fichiers du projet	7
5	La base de données	7
5.1	Le schéma	7
5.2	Où se trouve la base	8
5.3	Réglages de connexion	8
5.4	Conventions de lecture	8
6	Connecteurs et collecte	8
6.1	Le registre	8
6.2	Écrire un connecteur	8
6.3	Le moteur	8
6.4	Le journal	9
6.5	Les sources qui résistent	9
7	L'interface	9
7.1	Les cinq modules	9
7.2	Les filtres en cascade	9
7.3	Superposition et base 100	9
7.4	La carte	9
7.5	Le bilinguisme	9
7.6	La charte et l'adaptation aux écrans	10
8	Import, définitions et recherche	10
8.1	Import de fichiers	10
8.2	Les définitions	10
8.3	GoogleOPESc+	10
9	Tests	10
10	Mise en ligne et déploiement	11
10.1	L'hébergement	11
10.2	Préparer	11
10.3	Publier	11
10.4	Mettre à jour	11
10.5	Le serveur est en lecture seule	11
11	Comment la plateforme a été construite	11
11.1	Le point de départ	12
11.2	Le catalogue avant le code	12
11.3	Les connecteurs, un par un	12
11.4	Ce qui a résisté le plus longtemps	12
11.5	L'interface, par couches	12

11.6 Le bilinguisme, plus long que prévu	12
11.7 La charte, deux fois	12
11.8 La mise en ligne	12
12 Maintenance courante	13
12.1 Ajouter un indicateur	13
12.2 Ajouter un fournisseur	13
12.3 Régénérer les documents	13
12.4 Ce qui reste à faire	13

Chapitre 1

Vue d'ensemble

1.1 Ce que fait le programme

OPESc+ interroge les interfaces publiques de trois fournisseurs, harmonise ce qu'elles renvoient, écrit dans une base SQLite unique, et expose cette base par une interface Shiny en lecture seule.

Environ 6 600 lignes de R réparties en 25 fichiers, plus 1 100 lignes de tests. Le tout forme un paquet R installable, organisé selon la structure golem.

1.2 Les quatre étages

1. **Connecteurs** : une fonction par fournisseur, qui interroge et normalise. `connecteurs.R`
2. **Moteur de collecte** : parcourt le catalogue, appelle le connecteur voulu, écrit en base, journalise. `collecte.R`
3. **Interface** : cinq modules Shiny qui lisent la base et n'y écrivent jamais. `mod_*.R`
4. **Exports** : classeurs et images autonomes portant leurs sources. `exports.R`

Cette séparation a une conséquence pratique : une session de consultation ne peut pas altérer les données, et une collecte en cours n'empêche pas la consultation.

1.3 Le catalogue, pivot du programme

Rien n'est codé en dur. `inst/extdata/catalogue.csv` décrit chaque indicateur : code interne, catégorie, libellé, source, code employé chez le fournisseur, fréquences, unité, dimension pays.

Ajouter un indicateur d'un fournisseur déjà branché ne demande aucune ligne de code : une ligne dans ce fichier suffit.

Le code interne et le code source

Le *code source* est celui du fournisseur et lui appartient. Le *code interne* le préfixe de sa catégorie, par exemple C02.NY.GDP.MKTP.CD. Cette distinction permet qu'un même indicateur figure dans deux catégories sans collision, et rend les jointures sans ambiguïté.

1.4 Pourquoi R et Shiny

Une première version avait été construite en Django, cadre conçu pour des applications à comptes utilisateurs, formulaires et écritures concurrentes. OPESc+ n'est rien de tout cela : c'est un explorateur de données en lecture seule. Sur les quatre grandes fonctions de Django, une et demie servaient.

R présente en outre l'avantage d'être la langue de travail des statisticiens de la division. Une évolution future, projection ou comparaison, s'y écrit en quelques lignes là où elle demanderait une bibliothèque entière ailleurs.

Chapitre 2

Comment fonctionne Shiny

Ce chapitre est destiné à qui n'a jamais écrit d'application Shiny. Il explique le modèle, non l'ensemble de l'interface.

2.1 Deux fonctions, une conversation

Une application Shiny tient en deux fonctions. `ui` décrit ce que le navigateur affiche, `server` décrit ce qui se passe quand l'utilisateur agit. Shiny établit une liaison permanente entre les deux, par une connexion ouverte pendant toute la session.

```
ui <- fluidPage(
  selectInput("pays", "Pays", choices = c("CMR", "NGA")),
  plotOutput("courbe"))
```

```
server <- function(input, output, session) {
  output$courbe <- renderPlot({
    tracer(lire_serie(input$pays))
  })
}
```

L'utilisateur change le pays, Shiny s'en aperçoit, réexécute le bloc de `renderPlot` et renvoie l'image. Aucun code ne dit *quand* redessiner : c'est déduit des dépendances.

2.2 La réactivité

C'est le concept central, et le seul qui demande un effort. Shiny observe quelles valeurs un bloc de code consulte, et le réexécute quand l'une d'elles change. On ne programme pas la séquence, on déclare les dépendances.

Trois familles d'objets.

Les entrées, `input$quelque_chose`, viennent du navigateur. Lire une entrée crée une dépendance.

Les expressions réactives, `reactive()`, calculent une valeur intermédiaire. Elles sont paresseuses, elles ne s'exécutent que si quelqu'un les lit, et mémorisées, elles ne recalculent que si une dépendance a changé. C'est ce qui évite d'interroger la base dix fois pour un seul filtre.

Les observateurs, `observe()` et `observeEvent()`, produisent un effet sans rendre de valeur : mettre à jour un champ, écrire un message. Ils s'exécutent toujours, sans être lus.

Le piège de la dépendance involontaire

Un bloc dépend de *tout* ce qu'il lit. Un observateur qui consulte cinq entrées se réexécute au changement de n'importe laquelle. `isolate()` permet de lire une valeur sans en dépendre, et `observeEvent()` de ne réagir qu'à un déclencheur nommé, en ignorant le reste.

Dans le tableau de bord, c'est ce qui permet au graphique de ne se redessiner qu'au clic sur "Appliquer", et non à chaque frappe dans un champ.

2.3 Les sorties

Chaque `output$nom` est associé à une fonction de rendu : `renderPlot` pour une image, `renderUI` pour du balisage, `DT::renderDT` pour un tableau, `plotly::renderPlotly` pour un graphique interactif. Chacune a son correspondant côté interface, `plotOutput`, `uiOutput`, et ainsi de suite.

`renderUI` mérite une mention : il produit de l'interface à la volée. C'est puissant, mais coûteux, chaque rendu reconstruisant le balisage et le renvoyant au navigateur. La plateforme l'emploie là où la structure dépend des données, et l'évite ailleurs.

2.4 Les modules

Une application qui grandit finit par avoir des identifiants qui se marchent dessus : deux onglets qui déclarent chacun un champ `categorie`. Le module résout cela par un préfixe.

```
mod_tableau_bord_ui <- function(id) {
  ns <- NS(id)
  selectInput(ns("categorie"), "Catégorie", choices = NULL)
}

mod_tableau_bord_server <- function(id, con) {
  moduleServer(id, function(input, output, session) {
    ns <- session$ns
    observeEvent(input$categorie, { ... })
  })
}
```

À l'intérieur du module, `input$categorie` désigne le champ local ; à l'extérieur, il s'appelle `tableau_bord-categorie`. Les cinq onglets de la plateforme sont cinq modules.

L'erreur à ne pas refaire

Un module qui appelle `ns()` côté serveur doit d'abord faire `ns <- session$ns`. L'oubli produit *impossible de trouver la fonction ns* au moment du rendu, non au chargement : le message n'apparaît que lorsque l'utilisateur atteint la partie concernée, ce qui rend le diagnostic plus lent qu'il ne devrait.

2.5 Le cycle d'une session

À l'ouverture d'une page, Shiny exécute la fonction `ui` pour produire le document, puis la fonction `server` une fois par session. Les objets créés dans `server` appartiennent à cette session : deux visiteurs ne partagent rien, sinon la base.

À la fermeture de l'onglet, la session est détruite. Les connexions ouvertes dans `server` doivent être refermées, faute de quoi elles s'accumulent. La plateforme ouvre une connexion par session et la referme à la fin.

Ce qui s'exécute une fois et ce qui s'exécute souvent

Le code écrit directement dans `server` s'exécute une fois par session. Celui placé dans un `reactive` ou un `render` s'exécute chaque fois qu'une dépendance change. Placer une lecture coûteuse au mauvais endroit est la première cause de lenteur d'une application Shiny.

2.6 Les erreurs qui reviennent

Un if sur une valeur vide. `nzchar(NULL)` rend un vecteur vide, et `if` sur un vecteur vide lève une erreur. Une définition sans source a ainsi interrompu l'affichage de toute la recherche. La parade est `isTRUE()` ou l'opérateur `%%`.

Un rendu qui dépend d'une entrée pas encore créée. Au premier affichage, les champs mis à jour par le serveur sont vides. `req()` suspend proprement le rendu jusqu'à ce qu'ils soient renseignés.

Un script chargé avant Shiny. Les fichiers JavaScript placés dans l'en-tête s'exécutent avant le script de Shiny : y tester l'existence de l'objet `Shiny` revient à ne rien enregistrer. L'écouteur se pose sur le document, qui existe toujours.

2.7 Ce que Shiny ne fait pas

Il ne persiste rien. Chaque session repart de zéro, et l'état vit dans des objets réactifs détruits à la fermeture. Ce qui doit survivre va en base.

Il n'aime pas les traitements longs : pendant qu'un calcul tourne, la session ne répond plus. C'est pourquoi la collecte, qui dure des minutes, s'exécute en console et non depuis l'interface en production.

Chapitre 3

La structure golem

3.1 Un paquet plutôt qu'un script

`golem` impose d'organiser l'application comme un paquet R : un dossier `R/` pour le code, `inst/` pour les ressources, `tests/` pour les tests, un `DESCRIPTION` qui déclare les dépendances.

L'intérêt n'est pas cosmétique. Les fonctions sont documentées et testables une à une, les dépendances sont déclarées explicitement, et le tout s'installe ou se déploie comme n'importe quel paquet.

3.2 Les fichiers d'ossature

Fichier	Rôle
<code>app_ui.R</code>	Bandeau, barre d'onglets, thème, assemblage des modules
<code>app_server.R</code>	Connexion à la base, langue de session, appel des serveurs de modules
<code>run_app.R</code>	Point d'entrée exporté
<code>app_config.R</code>	<code>app_sys()</code> , qui retrouve un fichier dans <code>inst/</code>
<code>app.R</code>	Point d'entrée de l'hébergeur, à la racine du dépôt

Table 3.1: L'ossature de l'application

`app_sys()` appelle `system.file()` : en développement, `pkgload` le détourne vers `inst/` du projet ; une fois installé, il pointe vers le paquet. Le même code fonctionne dans les deux cas.

Chapitre 4

Les fichiers du projet

Fichier	Lignes	Rôle
<code>connecteurs.R</code>	950	Un connecteur par fournisseur, et le registre qui les associe aux sources
<code>mod_tableau_bord.R</code>	730	Filtres en cascade, graphiques, carte, projections, exports
<code>collecte.R</code>	560	Moteur de collecte, journal, compteurs, rattrapage
<code>import_manuel.R</code>	510	Import de fichiers, création d'indicateurs, suppression
<code>recherche_web.R</code>	500	Moteur de recherche, glossaire, correspondance des notions
<code>mod_recherche.R</code>	435	Interface de GoogleOPEsc+
<code>bd.R</code>	400	Schéma, connexion, lectures, diagnostics
<code>graphique_accueil.R</code>	375	Graphique SVG de la bannière, dormant
<code>definitions.R</code>	275	Récupération des définitions et de leur source
<code>mod_accueil.R</code>	245	Bannière, chiffres, cartes de fonctionnalités
<code>mod_collectes.R</code>	230	Import, suppression, journal
<code>publication.R</code>	230	Base compacte pour la mise en ligne
<code>graphiques.R</code>	180	Construction des graphiques
<code>mod_base_donnees.R</code>	170	Consultation tabulaire et export
<code>i18n.R</code>	160	Bilinguisme
<code>carte.R</code>	145	Carte choroplèthe et fiche pays

Table 4.1: Les fichiers principaux, par taille

Les autres sont brefs : `config.R` centralise les constantes, `icones.R` dessine les icônes en SVG, `pays_extra.R` complète les codes pays, `exports.R` produit les classeurs.

Chapitre 5

La base de données

5.1 Le schéma

Cinq tables.

`categorie` et `pays` sont des référentiels, chargés depuis `inst/extdata`. `indicateur` porte le catalogue et deux compteurs, nombre d'observations et date de dernière collecte. `observation` contient les données. `definition` porte les définitions, leur source et leur langue.

```
CREATE TABLE observation (
  code_interne TEXT NOT NULL,
  iso3         TEXT NOT NULL,
  frequence    TEXT NOT NULL,
  date_periode TEXT NOT NULL,
  annee        INTEGER,
  valeur       REAL,
  PRIMARY KEY (code_interne, iso3, frequence, date_periode))
```

Pourquoi une clé sur quatre colonnes

Elle rend l'écriture idempotente : relancer une collecte met à jour les valeurs révisées sans créer de doublons, par un `ON CONFLICT DO UPDATE`. C'est ce qui permet de relancer une collecte interrompue sans précaution.

5.2 Où se trouve la base

`chemin.base()` cherche dans cet ordre : la variable d'environnement `OPESC_BASE`, le fichier de configuration, la base de travail de l'utilisateur dans son dossier de données, puis celle livrée dans le paquet.

L'ordre compte. La base de travail passe avant celle du paquet : sans cela, dès qu'une base publiée existe dans `inst/extdata`, la plateforme cesse d'utiliser la base locale. Sur le serveur, le dossier utilisateur n'existe pas et la base livrée est retenue. C'est ce qui fait fonctionner le même code en local et en ligne.

5.3 Réglages de connexion

`connexion()` applique quatre réglages SQLite. Le journal en mode WAL autorise la lecture pendant l'écriture, la plateforme reste donc consultable pendant une collecte. La synchronisation relâchée multiplie par cinq à dix le débit d'écriture. Un cache élargi et des tables temporaires en mémoire accélèrent les tris des exports.

5.4 Conventions de lecture

Une valeur manquante n'est jamais enregistrée : l'absence de ligne vaut absence de donnée, jamais zéro. Les fournisseurs renvoient pourtant une ligne par couple pays-période, y compris vide ; sur un indicateur à faible couverture, plus de neuf dixièmes du volume seraient des lignes creuses.

Toute observation est ramenée au premier jour de la période qu'elle couvre. Cette convention permet de superposer des séries de pas différents sur un même graphique.

Chapitre 6

Connecteurs et collecte

6.1 Le registre

REGISTRE associe chaque valeur de la colonne `source` du catalogue à une fonction. Un indicateur dont la source n'y figure pas est désactivé au chargement plutôt que laissé visible et muet dans l'interface.

```
REGISTRE <- list(
  "Banque mondiale (WDI)" = connecteur_banque_mondiale,
  "FMI (WEO)"             = connecteur_fmi_weo,
  ...)
```

6.2 Écrire un connecteur

Une fonction qui reçoit un code source, des bornes facultatives, et rend un tableau à quatre colonnes : `iso3`, `date_période`, `frequence`, `valeur`. Le moteur se charge du reste, y compris de l'écriture et du journal.

Ce qu'un connecteur ne doit pas faire

Ni écrire en base, ni supposer que la réponse est complète, ni échouer bruyamment sur une série absente. Un fournisseur qui renomme un code doit produire une série vide et un message, non une interruption qui arrêterait toute la collecte.

6.3 Le moteur

`collecter()` parcourt les indicateurs, appelle le connecteur, écrit par lots, et tient le journal. Une fonction d'avancement est passée en argument, ce qui permet d'afficher la progression en console comme dans l'interface sans dupliquer le moteur.

`collecter_manquants()` restreint aux indicateurs sans données. Ceux dont la source n'a pas de connecteur ne sont pas tentés : ce serait remplir le journal d'échecs prévisibles. Ils sont recensés en fin d'exécution.

6.4 Le journal

Chaque exécution est tracée : portée, déclenchement, statut, valeurs ajoutées et révisées, erreurs. Une collecte peut se terminer sans erreur et sans rien ramener, par exemple si un fournisseur a renommé un code ; sans journal, cette panne silencieuse passerait inaperçue des mois et les analyses reposeraient sur des séries figées.

6.5 Les sources qui résistent

Deux pages construisent leur lien de téléchargement en JavaScript : celle des prix des produits de base et celle de l'Atlas de Harvard. Aucune adresse ne figure dans le code de la page, et aucun programme ne peut la trouver. Six voies ont été essayées sur les produits de base avant d'en tirer la conclusion : paquet dédié, requêtes ciblées en SDMX, flux entier, classeur de la Banque mondiale, et deux variantes de clé. L'import manuel prend le relais, et c'est un choix assumé plutôt qu'un pis-aller.

Chapitre 7

L'interface

7.1 Les cinq modules

Un par onglet. Chacun reçoit la connexion et n'expose que son interface et son serveur.

7.2 Les filtres en cascade

Catégorie, indicateur, fréquence, période, pays. Chaque étape ne propose que ce qui existe réellement en base pour la sélection courante. Aucun filtre ne mène à un graphique vide, ce qui est la règle de conception la plus structurante du tableau de bord.

Techniquement, chaque étape est un `observeEvent` sur l'étape précédente, qui interroge la base et met à jour le champ suivant par `updateSelectizeInput`.

Le piège des options de mise à jour

Passer `options` à `updateSelectizeInput` réinitialise le composant côté navigateur, qui perd alors son caractère multiple. Le champ redevenait mono-sélection dès le premier chargement, et il devenait impossible de choisir deux indicateurs. Les options se posent à la création, jamais à la mise à jour.

7.3 Superposition et base 100

Jusqu'à six séries. Quand les unités diffèrent, elles sont ramenées en base 100 à leur première observation : superposer un pourcentage du produit intérieur brut et un cours en dollars par tonne sur un même axe écraserait l'un des deux.

7.4 La carte

Une carte choroplèthe produite par plotly, bornée aux centiles 2 et 98 de la distribution. Sans cette précaution, quelques valeurs extrêmes donneraient à tous les autres pays la même teinte. Les agrégats en sont exclus, comme des classements.

7.5 Le bilinguisme

`tr()` traduit par recherche dans un dictionnaire de plus de cinq cents entrées, chargé depuis un CSV. Le changement de langue recharge la page avec un paramètre d'adresse, ce qui permet de traduire jusqu'aux libellés construits au démarrage.

Deux pièges. Les constantes évaluées au chargement du paquet ne peuvent pas appeler `tr()`, qui n'existe pas encore : elles passent par le dictionnaire au moment du rendu. Et un test balaie le code à la recherche de textes accentués hors de toute portée `tr()`, car un remplacement de bloc fait disparaître un `tr()` sans que rien ne le signale. Ce test est né de quatre régressions successives.

7.6 La charte et l'adaptation aux écrans

Onze variables CSS portent toute la charte : trois bleus, un rouge réservé aux actions, les gris et les bordures. Changer la charte revient à changer ces onze valeurs.

Trois seuils d'adaptation, à 1100, 820 et 560 pixels. Une règle générale empêche le document de défiler horizontalement : les éléments qui débordent le font dans leur propre cadre, ce qui évite qu'un tableau ou une barre d'onglets ne décale toute la page.

Chapitre 8

Import, définitions et recherche

8.1 Import de fichiers

L'onglet Collectes accepte un CSV ou un classeur. La lecture est tolérante : les colonnes sont repérées par leur contenu autant que par leur intitulé, et le format large, une colonne par période, est reconnu, la fréquence étant lue dans le nom de la colonne.

Deux modes. *Compléter* ajoute ou corrige. *Remplacer* efface d'abord chaque série concernée, ce qui évite qu'un fichier plus court laisse subsister d'anciennes valeurs au-delà de sa dernière date, donnant une série qui paraît complète alors qu'elle mélange deux millésimes.

Une donnée importée vaut une donnée collectée

Elle emprunte la même fonction d'écriture et alimente les mêmes compteurs. Un indicateur est actif s'il peut être alimenté *ou s'il l'est déjà* : sans cette seconde condition, une série importée resterait invisible faute de connecteur.

8.2 Les définitions

Elles viennent du fournisseur, avec le nom de l'organisme qui en répond. La Banque mondiale publie pour chaque indicateur un texte et une attribution : les deux sont repris tels quels, en anglais.

L'interface préfère le glossaire français quand la notion y figure. Vingt-neuf notions portent des variantes de libellé, car un indicateur s'appelle "Taux de croissance du PIB réel", jamais "Croissance économique". Les variantes sont éprouvées de la plus longue à la plus courte, pour que la plus spécifique l'emporte.

Le seuil est volontairement strict : deux mots partagés au moins. Un seul suffisait, et "prix" rattachait "PIB par habitant, prix courants" à l'indice des prix à la consommation. Une définition fausse est pire qu'une définition absente.

8.3 GoogleOPESc+

Une recherche restreinte à trente-cinq sources choisies. Sans configuration, chaque lien lance la recherche du terme sur le site visé, ce qui garantit l'absence de hors-sujet. Avec un moteur de recherche programmable, les résultats s'affichent directement, filtrés une seconde fois sur la liste des domaines par précaution.

Chapitre 9

Tests

Dix-neuf fichiers, environ 1 100 lignes. Ils ne vérifient pas seulement que le code s'exécute, mais que les règles de lecture tiennent : agrégats exclus des cartes, séries lacunaires non interpolées, aucun texte visible sans traduction, aucune définition sans source, tout module appelant `ns()` le définissant.

Plusieurs sont nés d'erreurs constatées et les rendent impossibles à reproduire. C'est leur meilleur usage : un test écrit après coup vaut mieux qu'un test écrit par principe.

```
testthat::test_local()
```

Chapitre 10

Mise en ligne et déploiement

10.1 L'hébergement

Posit Connect Cloud, offre gratuite, déploiement depuis un dépôt GitHub public avec redéploiement automatique à chaque envoi.

10.2 Préparer

```
pkgload::load_all()
preparer_publication(annee_min = 1990)
verifier_publication()
rsconnect::writeManifest()
```

`preparer_publication()` construit une base compacte : les codes textuels sont remplacés par des entiers renvoyant à deux tables de correspondance, ce qui ramène la ligne de 150 à 34 octets. Une vue nommée `observation` rend l'opération transparente, le reste du code l'interrogeant sans savoir comment elle est rangée.

Deux contraintes à connaître

GitHub refuse un fichier au-delà de 100 méga-octets et en signale un au-delà de 50. L'argument `annee_min` limite la profondeur historique quand la compaction ne suffit pas.
Le dépôt est public : aucune clé ne doit y figurer. Le `.gitignore` écarte `.Renviron`, et `verifier_publication()` le contrôle avant l'envoi.

10.3 Publier

Créer un dépôt GitHub public, y pousser le projet, puis sur `connect.posit.cloud` choisir le dépôt, la branche et `app.R` comme fichier principal. Déclarer ensuite les variables d'environnement dans les réglages du contenu.

`app.R` charge le paquet depuis les sources plutôt que de l'installer : `opesplus` n'est publié sur aucun dépôt de paquets, et l'hébergeur ne saurait pas où le prendre. Les appels à `library()` qu'il contient ne servent pas au fonctionnement, le code étant préfixé par les espaces de noms, mais à `writeManifest()`, qui établit la liste des dépendances en lisant ce fichier.

10.4 Mettre à jour

```
preparer_publication(annee_min = 1990)
git add . && git commit -m "... " && git push
```

La base ne se met pas à jour toute seule

C'est l'oubli le plus fréquent. Pousser du code sans relancer `preparer_publication()` publie l'ancienne base : l'interface évolue, les données non.

10.5 Le serveur est en lecture seule

L'onglet Collectes n'y montre que le journal. La collecte et l'import restent des opérations locales, suivies d'une republication. Le contenu se met en veille après inactivité : le premier visiteur attend une trentaine de secondes, ce qu'il faut savoir avant une démonstration.

Chapitre 11

Comment la plateforme a été construite

Ce chapitre retrace la construction. Il n'a pas d'utilité opérationnelle, mais il explique des choix qui paraîtraient arbitraires sans lui.

11.1 Le point de départ

Une première version existait en Django. Elle fonctionnait, mais l'écart entre l'outil et le besoin était grand : Django sert à bâtir des applications à comptes, formulaires et écritures concurrentes, quand OPESc+ est un explorateur en lecture seule. Le portage en R et Shiny a été décidé pour cette raison, et parce que R est la langue de travail de la division.

11.2 Le catalogue avant le code

Le premier travail n'a pas été d'écrire des fonctions, mais de dresser le catalogue : quels indicateurs, chez quel fournisseur, sous quel code, à quelle fréquence. Ce fichier a précédé tout le reste, et c'est lui qui a dicté l'architecture. Un programme qui lit un catalogue se modifie en modifiant le catalogue.

11.3 Les connecteurs, un par un

Chaque fournisseur a demandé son propre travail d'exploration : trouver l'adresse, comprendre le format, identifier les codes. Les interfaces publiques sont documentées inégalement, et certaines le sont mal.

Les portails changent

Le Fonds monétaire international a réorganisé ses interfaces deux fois pendant la construction. Une adresse a été retirée, une autre s'est mise à refuser toute requête automatisée. L'Atlas de Harvard a changé de technologie d'accès.

La leçon en a été tirée : chaque source critique dispose de plusieurs voies, et le connecteur bascule de l'une à l'autre en le signalant. C'est pourquoi le connecteur des produits de base en compte quatre.

11.4 Ce qui a résisté le plus longtemps

Les prix des produits de base. Six approches ont été tentées : le flux SDMX en CSV, le même en JSON, le classeur Pink Sheet de la Banque mondiale, un paquet R dédié, des requêtes ciblées, puis le flux entier filtré côté client.

Trois obstacles se sont succédé. L'en-tête demandé faisait échouer la requête sans erreur, ce qui a fait croire à un blocage. L'ordre des dimensions n'était pas celui que documentaient les exemples publiés. Et le joker s'écrit différemment selon la version du protocole.

La solution retenue est l'import manuel d'un fichier téléchargé. Ce n'est pas un aveu d'échec : la page construit son lien en JavaScript, aucun programme ne peut le trouver, et s'obstiner aurait produit un code fragile.

11.5 L'interface, par couches

Le tableau de bord d'abord, puisqu'il porte l'essentiel de l'usage. Puis la base de données, l'accueil, les collectes. GoogleOPESc+ est venu en dernier, d'un besoin exprimé en cours de route : trouver une donnée ou une publication sans se perdre dans des résultats dont on ignore la provenance.

11.6 Le bilinguisme, plus long que prévu

Traduire une interface ne consiste pas à traduire des chaînes. Il a fallu traiter les libellés construits au démarrage, ceux qui viennent de fichiers de données, ceux qui sont assemblés par concaténation. Quatre régressions successives ont fait disparaître des traductions sans que rien ne le signale, jusqu'à ce qu'un test balaie le code à la recherche de textes accentués hors de toute portée de traduction.

11.7 La charte, deux fois

Une première charte, bleu marine et doré, n'appartenait à personne. Une deuxième, tirée des armoiries de la République, s'est révélée fautive : le ministère emploie le bleu, le blanc et le rouge. La troisième est la bonne.

L'enseignement vaut d'être noté : les couleurs d'une institution se relèvent, elles ne se déduisent pas.

11.8 La mise en ligne

Deux obstacles. La base pesait plus de deux cents méga-octets quand GitHub en refuse cent : la compaction par entiers l'a ramenée à soixante. Et le dépôt étant public, aucune clé ne pouvait y figurer, d'où les variables d'environnement déclarées chez l'hébergeur.

Chapitre 12

Maintenance courante

Commande	Usage
<code>diagnostic_plateforme()</code>	État de la base, par catégorie. À lancer en premier devant un écran vide
<code>collecter_manquants()</code>	Collecte tout ce qui n'a pas de données
<code>rafraichir_compteurs()</code>	Remet les compteurs en accord avec les observations
<code>marquer_langue_definitions()</code>	Renseigne la langue sans aucune requête
<code>diagnostic_frequences()</code>	Fréquences réellement présentes
<code>codes_categories()</code>	Codes des catégories et état de collecte
<code>verifier_recherche()</code>	État du moteur de recherche
<code>verifier_publication()</code>	Contrôles avant un envoi

Table 12.1: Les commandes de diagnostic

12.1 Ajouter un indicateur

Une ligne dans `catalogue.csv`, puis `preparer_base()` et une collecte sur sa catégorie. Aucune ligne de code si le fournisseur est déjà branché. Penser à ajouter le libellé au dictionnaire de traduction.

12.2 Ajouter un fournisseur

Écrire le connecteur, l'inscrire au registre, ajouter les indicateurs au catalogue. Six fournisseurs restent à brancher : CNUCED, FAO, OIT, OCDE, PNUD, Transparency International.

12.3 Régénérer les documents

Les sources sont dans `dev/manuel`. Le PDF passe par LaTeX, le Word par la bibliothèque `docx`, et les deux partagent le même contenu, lu dans les mêmes fichiers JSON.

```
python3 gen_latex.py    # manuel, PDF
node gen.js            # manuel, Word
python3 gen_note.py    # note de présentation
python3 gen_guide.py   # ce guide
```

Effacer les fichiers auxiliaires de LaTeX entre deux compositions, sans quoi le sommaire garde une numérotation périmée.

12.4 Ce qui reste à faire

Brancher les six fournisseurs manquants. Automatiser la collecte par une tâche planifiée hebdomadaire suivie d'une republication. Traduire les libellés des indicateurs en anglais dans l'interface, le travail étant fait pour le manuel. Et décider du régime d'hébergement si la plateforme doit s'ouvrir plus largement.